

# User Flow: Login to Booking PG Successfully

Complete Technical Defense, Architecture, Screen Code Walkthrough, Slide Script & Viva Q&A;

|                                                          |                                     |                                               |
|----------------------------------------------------------|-------------------------------------|-----------------------------------------------|
| Assigned Role: User Journey (Login to Confirmed Booking) | Target Stack: Flutter Mobile (Dart) | Target Pattern: Reactive State & Safe Routing |
|----------------------------------------------------------|-------------------------------------|-----------------------------------------------|

## 1. High-Level Architecture & User Flow Pipeline

The user flow represents the core business transaction in PG Finder: taking a student or tenant from secure entry to discovering suitable accommodation, tailoring tenancy preferences, calculating costs live, and finalizing an authorized booking with zero navigation traps.

| Step & Screen                         | Source File                             | Core Function & Responsibility                                                             | Next Target Route                            |
|---------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------|----------------------------------------------|
| 1. Authentication<br>Login Screen     | login_screen.dart                       | Regex email/username checks, credential verification, and role differentiation.            | BottomNavScreen (HomeScreen)                 |
| 2. Discovery<br>Home & Explore        | home_screen.dart<br>bottom_nav_bar.dart | City selection, category filters (Boys/Girls/Co-ed), search bar, and PG cards.             | PgDetailScreen(pg)                           |
| 3. Inspection<br>PG Detail Screen     | pg_detail_screen.dart                   | Display gallery image, amenities (Food, AC, WiFi), rent, and Call/WhatsApp CTAs.           | BookVisitScreen(pg)                          |
| 4. Customization<br>Book Visit Screen | book_visit_screen.dart                  | Native date picker, stay duration dropdown, guest counter, and real-time cost calculation. | PaymentScreen (pg, total, duration, members) |
| 5. Payment & Review<br>Payment Screen | payment_screen.dart                     | Trust shield banner, transparent bill breakdown, Pay at Check-in option selection.         | PaymentSuccessScreen (pushReplacement)       |
| 6. Confirmation<br>Payment Success    | payment_success_screen .dart            | Success badge, generated Transaction ID, 3-step 'What's Next?' roadmap.                    | pushAndRemoveUntil (Bookings or Home tab)    |

## 2. Deep-Dive Code Walkthrough (Screen-by-Screen)

### Step 1: Login & Role-Based Authentication (login\_screen.dart)

**Architectural Concept:** Manages user input state using GlobalKey<FormState> and TextEditingController. Validates input formatting through regular expressions and routes users based on their assigned role.

- **Input Validation:** Ensures emails match standard format `^[lw-\.]+\@([\w-]+\.)+[lw-]{2,4}$` and passwords meet minimum length.
- **Role Separation:** Prevents unauthorized escalation by routing admin13 to Admin Dashboard, organizer13 to Organizer

Dashboard, and regular tenants to BottomNavScreen().

```
// In lib/screens/auth/login_screen.dart
void _handleLogin() {
  if (!_formKey.currentState!.validate()) return;
  final loginInput = _emailController.text.trim().toLowerCase();
  final password = _passwordController.text;

  if (!_checkCredentials(loginInput, password)) {
    setState(() => _errorMessage = 'Incorrect credentials. Try again.');
```

```
    return;
  }
}
```

```
Widget nextScreen;
```

```
if (loginInput == 'admin13' || loginInput == 'admin@pgfindar.com') {
```

```
  nextScreen = const AdminDashboardPage();
```

```
} else {
```

```
  nextScreen = const BottomNavScreen(); // Default user dashboard
```

```
}
```

```
Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => nextScreen));
```

```
}
```

## Step 2: Home Dashboard & PG Exploration (home\_screen.dart & bottom\_nav\_bar.dart)

**Architectural Concept:** The root scaffold utilizes an IndexedStack. This maintains all tab widgets in memory, avoiding redundant rebuilds and preserving the tenant's exact scroll and filter position.

- **Interactive Filters:** City dropdown (Rajkot, Ahmedabad), search query bar, and quick filter chips (Boys, Girls, Co-ed).
- **Model Forwarding:** Tapping 'Book Now' packages the active PGAccommodation model object and passes it to PgDetailScreen.

```
// In lib/widgets/bottom_nav_bar.dart & home_screen.dart
IndexedStack(index: _currentIndex, children: _screens); // Tabs: Home, Saved, Bookings, Profile

// Trigger on PG Card 'Book Now' Button:
ElevatedButton(
  onPressed: () {
    Navigator.push(context, MaterialPageRoute(
      builder: (context) => PgDetailScreen(pg: pg),
    ));
  },
  child: const Text('Book Now'),
)
```

### Step 3: Property Details & Decision Inspection (pg\_detail\_screen.dart)

**Architectural Concept:** Provides zero-overflow proportional layout displaying image gallery, rating badges, facilities chips, and communication channels without requiring external dependencies.

- **Defensive Data Binding:** Employs safe fallbacks (e.g. `pg?.name ?? 'Green Valley PG'`) to guarantee zero UI crashes.
- **External Contact Actions:** Connects users directly to owners via simulated Call and WhatsApp hooks.
- **Primary CTA:** 'Book a PG' pushes the `BookVisitScreen` with the property instance.

```
// In lib/screens/pg_detail_screen.dart
ElevatedButton(
  onPressed: () {
    Navigator.push(context, MaterialPageRoute(
      builder: (context) => BookVisitScreen(pg: pg),
    ));
  },
  style: ElevatedButton.styleFrom(backgroundColor: const Color(0xFF10B981)),
  child: const Text('Book a PG'),
)
```

### Step 4: Booking Customization & Reactive Calculation (book\_visit\_screen.dart)

**Architectural Concept:** Captures reservation variables and executes live recalculation of the total package price whenever duration or member counts update.

- **Date Safety:** `_pickDate()` launches `showDatePicker()` with `firstDate: DateTime.now()`, forbidding past dates.
- **Reactive Pricing Formula:** Dynamically translates duration strings ('1 Month', '3 Months', '6 Months', '1 Year') into multiplier constants (\$1, 3, 6, 12\$) and calculates:  $\text{totalPrice} = \text{monthlyPrice} * \text{multiplier} * \text{\_members}$ .

```
// In lib/screens/book_visit_screen.dart
int _calculateTotalPrice(int monthlyPrice) {
  int multiplier = 6;
  if (_selectedDuration == '1 Month') multiplier = 1;
  if (_selectedDuration == '3 Months') multiplier = 3;
  if (_selectedDuration == '6 Months') multiplier = 6;
  if (_selectedDuration == '1 Year') multiplier = 12;
  return monthlyPrice * multiplier * _members;
}

// Confirm Booking Button:
Navigator.push(context, MaterialPageRoute(
  builder: (context) => PaymentScreen(
    pg: widget.pg, totalAmount: totalPrice,
    duration: _selectedDuration, members: _members,
  ),
));
```

### Step 5: Checkout & Billing Review (payment\_screen.dart)

**Architectural Concept:** Establishes billing clarity with a transparent financial breakdown (Rent, Maintenance, Security Deposit) and sets payment mode to 'Cash on Delivery / Pay at Check-in'.

- **Trust Badge:** Displays an emerald 'Your booking is safe and secure' shield banner.
- **Anti-Double Submission:** Uses `Navigator.pushReplacement` to eliminate the checkout form from the backstack once confirmed.

```
// In lib/screens/payment_screen.dart
Navigator.pushReplacement(context, MaterialPageRoute(
  builder: (context) => PaymentSuccessScreen(
    pg: widget.pg, totalAmount: widget.totalAmount,
    duration: widget.duration, members: widget.members,
  ),
));
```

## Step 6: Confirmation & Clean Stack Reset (payment\_success\_screen.dart)

**Architectural Concept:** Delivers confirmation feedback with an animated checkmark, unique transaction receipt code, and an explicit 3-step 'What's Next?' timeline. Clears all navigation history using pushAndRemoveUntil.

```
// In lib/screens/payment_success_screen.dart
// Button 1: 'View booking' -> Directly opens My Bookings Tab (index 2)
Navigator.pushAndRemoveUntil(context, MaterialPageRoute(
  builder: (context) => const BottomNavScreen(initialIndex: 2),
), (route) => false); // Clears all prior screens

// Button 2: 'Back to home' -> Directly returns to Home Tab (index 0)
Navigator.pushAndRemoveUntil(context, MaterialPageRoute(
  builder: (context) => const BottomNavScreen(initialIndex: 0),
), (route) => false);
```

### 3. State & Parameter Lifecycle Matrix

This table traces how critical variables originate, mutate, and travel across the widget hierarchy:

| Variable / State  | Data Type           | Originating Screen          | Consumer Screens                    | Lifecycle Purpose                                                    |
|-------------------|---------------------|-----------------------------|-------------------------------------|----------------------------------------------------------------------|
| pg                | PGAccommodation?    | home_screen.dart            | Detail, BookVisit, Payment, Success | Carries property title, address, monthly rate, and image asset path. |
| _selectedDuration | String ('6 Months') | book_visit_screen.dart      | PaymentScreen, SuccessScreen        | Calculates lease duration multiplier and prints lease summary.       |
| _members          | int (default: 1)    | book_visit_screen.dart      | PaymentScreen, SuccessScreen        | Multiplies room capacity cost and specifies guest count.             |
| totalAmount       | int (calculated)    | book_visit_screen.dart      | PaymentScreen, SuccessScreen        | Total payable rupee amount computed by calculation engine.           |
| transactionId     | String ('TXN...')   | payment_success_screen.dart | PaymentSuccessScreen                | Acts as the unique receipt reference for physical check-in.          |

### 4. Slide-by-Slide Presentation Speech Script

Use this structured script to present your role with confidence and clarity in front of evaluators:

| Slide / Step                           | Exact Speaking Script (What to Say)                                                                                                                                                                                         | Key Technical Keywords                                      |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Slide 1<br>Introduction                | "Respected evaluators, today I am presenting the core end-to-end user booking flow of PG Finder — taking a tenant seamlessly from login authentication to a fully finalized PG reservation."                                | User Journey, State Transition, Mobile UX                   |
| Slide 2<br>Authentication              | "Our flow starts at the Login Screen. We employ regex pattern matching and input validation to verify credentials. Our role-router directs standard users to their exploration dashboard while preserving system security." | GlobalKey<FormState>, Regex Validation, Role-Based Routing  |
| Slide 3<br>Discovery & Catalog         | "Upon entering the home dashboard, an IndexedStack preserves tab states. Users can browse verified PGs, filter by city or category, and inspect price badges before selecting 'Book Now'."                                  | IndexedStack, Dynamic Filtering, Model Forwarding           |
| Slide 4<br>Property Inspection         | "On the PG Details page, users evaluate accommodation photos, verified amenities like Wi-Fi and food, location, and owner contacts. Clicking 'Book a PG' launches our booking customization screen."                        | Defensive Data Binding, Zero-Overflow UI                    |
| Slide 5<br>Customization & Calculation | "In the Book Visit screen, tenants choose their check-in date via a native date picker that forbids past dates. As they adjust duration and member count, our reactive engine dynamically updates the total price."         | showDatePicker, Dynamic Price Multiplier, Reactive setState |
| Slide 6<br>Checkout & Terms            | "The Payment screen provides complete billing transparency. To ensure student trust, we support 'Pay on Check-in'. We use pushReplacement to eliminate the checkout form from history once confirmed."                      | Itemized Billing, Cash on Delivery, pushReplacement         |
| Slide 7<br>Success & Stack Cleanliness | "Finally, the Payment Success screen confirms the reservation with an emerald checkmark, unique Transaction ID, and next-steps timeline. Using pushAndRemoveUntil, we reset the navigation stack safely."                   | pushAndRemoveUntil, Stack Hygiene, Receipt Timeline         |

## 5. Viva / Examiner Q&A; Defense Guide

Review these 8 high-frequency technical questions to score maximum marks during viva questioning:

### Q1: Why did you use `Navigator.pushReplacement` on the Payment Screen instead of standard `Navigator.push`?

Answer: When finalizing a booking, we must avoid the 'checkout loop'. If we used simple push, tapping the hardware back button from the success screen would return the user into an active checkout form. With `pushReplacement`, the `PaymentScreen` is destroyed and replaced by `PaymentSuccessScreen`.

### Q2: How does `Navigator.pushAndRemoveUntil` function on the Success Screen?

Answer: It completely purges the entire existing route stack `((route) => false)` and installs `BottomNavScreen` as the fresh root. This guarantees clean navigation hygiene so the back button exits the app rather than reopening old booking forms.

### Q3: How is dynamic pricing calculated in `book_visit_screen.dart`?

Answer: The `_calculateTotalPrice` function checks `_selectedDuration` and sets a multiplier: 1 month = 1, 3 months = 3, 6 months = 6, 1 year = 12. It then computes: `monthlyPrice * multiplier * _members`. Because this function is called inside `build()` and updated via `setState()`, the UI reacts instantly to user input.

### Q4: Why did you pick `IndexedStack` for the dashboard navigation?

Answer: Standard bottom navigation switches often dispose and recreate widgets on every tab change. `IndexedStack` keeps all 4 screens (Home, Saved, Bookings, Profile) alive in memory, preserving scroll offsets, user filter states, and loaded image caches.

### Q5: How does the app prevent booking dates in the past?

Answer: The `showDatePicker` API enforces `firstDate: DateTime.now()`. Any past calendar days are disabled and unselectable on the native platform date picker dialog.

### Q6: How does the application achieve role-based access control during login?

Answer: In `_handleLogin()`, after credentials match, the input username/email is evaluated against administrative patterns. Admin and Organizer accounts are dispatched to their specific dashboards, while tenants enter the general consumer flow.

### Q7: How is data passed across 5 consecutive screens without a state management package?

Answer: We use Constructor Parameter Injection. Each target screen's constructor strictly defines its expected inputs (e.g., `PGAccommodation pg`, `int totalAmount`, `String duration`). Flutter's declarative widget tree handles strongly-typed data handoff cleanly and efficiently.

### Q8: What design pattern did you adopt for the booking receipt?

Answer: A Stepper Timeline Pattern. The 'What's Next?' component shows completed, active, and upcoming stages (Booking Confirmed -> Owner Notified -> Pay at Check-in) so tenants know exactly what to expect upon arrival.